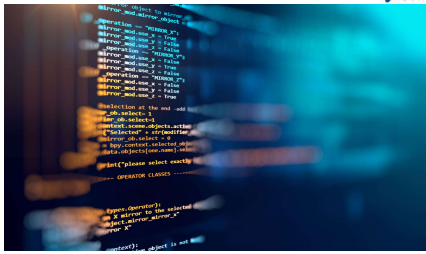




Python Programming Foundations

Week 4 - Debugging, Testing, & Reading Structured Code
Day 1: "Debugging as Evidence Gathering"

1

Review Coding Lab

2




A Bug Is Information

A bug is information, not a verdict.

When code breaks or gives the wrong answer, the program is giving you clues.

Today, practice debugging by asking:

- ▶ What did I expect?
- ▶ What actually happened?
- ▶ Where is the first useful clue?
- ▶ What fix does the evidence justify?

3




bug → information

A "bug" is "information"

4




Build Becomes Inspect

In earlier weeks, you built small programs.

Now you inspect behavior when a program does not do what it should.

Debugging uses familiar skills:

- reading code
- running code
- tracing values
- explaining output

5




Symptoms Show Up, Sources Hide

A symptom is what you notice first - The source is where the problem actually begins.

Examples:

- ▶ **symptom**: wrong final total
- ▶ **source**: tax calculated from the wrong value
- ▶ **symptom**: program will not run
- ▶ **source**: missing quote, colon, or parenthesis

6

Southwest Tech

The visible problem may start earlier in the program.

SYMPTOM

What we see

```

The program ran.
Here is the result:
X Total: -25 ← Wrong

```

The output is wrong, but the cause is not here.

SOURCE

Where it starts

```

Earlier in the program...
1 total = 0
2 for n in numbers:
3     value = n - 5 ← Bug here (off by 10)
4     total = total + value
5     print("Total:", total)

```

A small mistake here changes everything earlier.

Trace backward. Find the true source.

Symptoms Show Up, Sources Hide

7

Southwest Tech

Working baseline ->
compare structure ->
revise one meaningful thing ->
explain the improvement.

What Counts as Success Today

A useful revision should make the program:

clearer	easier to test	easier to change	easier to explain
---------	----------------	------------------	-------------------

8

What Counts as Success Today

- compare two approaches
- collect timing evidence
- describe growth cautiously
- use basic Big-O vocabulary
- identify a limitation of your evidence

9

Southwest Tech

What We Will Use Today

- ▶ expected versus actual
- ▶ tracebacks
- ▶ labeled print-debugging
- ▶ one-change-at-a-time repair
- ▶ simple evidence notes

These tools help you investigate *before* changing code.

10

Southwest Tech

Debugging Toolbox

Small tools. Clear thinking. Better programs.

expected vs actual

Define what should happen. Compare with what did.

```

Expected: 42
Actual: -2
Mismatch found.

```

traceback clue

Use the error message to find the failing point.

```

Clear
Line 17
In total_sum():
TypeError: unsupported...

```

labeled print

Print key values with labels to see what's happening.

```

INPUT n = 5
STEP value = -3
STEP total = -2

```

one-change repair

Make one small change. Test again. Confirm the fix.

```

Change:
value = n - 5
(was n - 5)

```

evidence notes

Record what you observed and what you changed.

```

Observed: total went negative.
Changed line 17.
Now correct.

```

Debugging is not guessing. It's collecting clues, making small changes, and learning.

Toolbox

11

Southwest Tech

What We Will "Skip" For Now

- ▶ full debugger workflows
- ▶ logging architecture
- ▶ pytest mastery
- ▶ profiling for now.

Today's required target is smaller:

- ▶ find useful evidence
- ▶ make a justified fix
- ▶ explain why the fix works

12



Parked for Later
Powerful topics. Useful later. Not our focus today.

full debugger workflows
Step through, inspect, and control complex programs.

logging architecture
Designing logs that are structured, consistent, and useful at scale.

pytest mastery
Fixtures, parametrization, marks, and advanced testing patterns.

profiling
Measure performance, find bottlenecks, improve efficiency.

We'll come back to these when you're ready. Strong foundations first.

Parked For Later

13



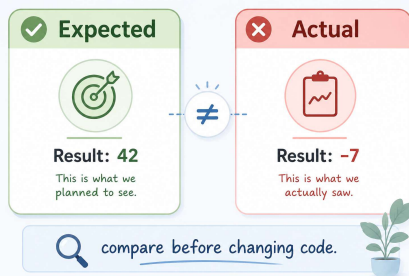
Expected Versus Actual

Expected behavior is what should happen.
Actual behavior is what happened when the program ran.

"Debugging" begins when you compare:

- ▶ expected output
- ▶ actual output
- ▶ the difference between them

14



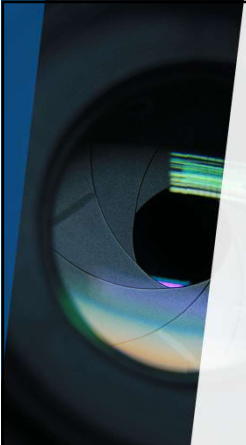
Expected
Result: 42
This is what we planned to see.

Actual
Result: -7
This is what we actually saw.

compare before changing code.

Expected vs. Actual

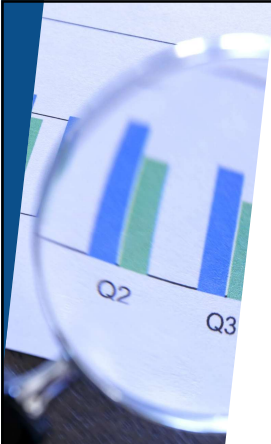
15



Demo #1

"Broken Syntax Example"

16



What to Watch For

A syntax error prevents Python from running the program.

Read the message for:

- ▶ file name
- ▶ line number
- ▶ error type
- ▶ the nearby code Python could not understand

17



Read the clues in a Python error

A few key parts tell you **where** and **what** went wrong.

```
Traceback (most recent call last):
  File "grades.py", line 14, in <module>
    average = total / count
ZeroDivisionError: division by zero
```

1 File name
Tells you which file has the problem.

2 Line number
Tells you where in the file it happened.

3 Error type
Tells you what kind of problem it is.

Use these clues to find the cause and fix the problem.

Demo 1: Syntax Error Signal

18



19



20

Discount Calculation

Item price: \$100
Discount: 20%

Expected
\$80.00
100 - 20% of 100 = 80

Actual
\$120.00
This doesn't look right.

Let's find where the calculation went wrong.

Calculation Steps

- 1 Start with item price 100.00
- 2 Calculate discount (20% of 100) 20.00
- 3 Apply discount to price
price = price - discount 120.00
- 4 Tax (0%) 0.00
- 5 Final price 120.00

Investigate here

The discount should be subtracted, not added. price = price - discount

Demo 2: Logic Bug
Expected vs Actual

21

Ask One Question,
Print One Clue

Print-debugging should answer a specific question.

Weak

```
1 print("here")
```

Better

```
2 print("subtotal before tax:", subtotal)
```

Every debug print should reveal a value or path you need to inspect.

22

Weak Print Debugging
Doesn't tell you what's happening.

Code

```
print("here")
```

Output

```
here
here
here
```

Not helpful. You have to guess.

Why it's weak

- You don't know what value this is, or where it came from.

Stronger Print Debugging
Answers a question.

Code

```
print("subtotal before tax:", subtotal)
```

Output

```
subtotal before tax: 87.58
```

Clear and useful. You learn something.

Why it's stronger

- Tells you what the value is.
- Shows the variable name or context.
- Helps you understand what's happening.

Good rule: Make your prints say something useful.

If someone else saw this output, would they understand it?

Ask One Question,
Print One Clue

23



24



What to Watch For

Watch where the grade summary first goes wrong.

Trace:

- ▶ starting student
- ▶ running total
- ▶ added score
- ▶ calculated average
- ▶ first incorrect value

25



Grade Calculation Trace

Follow the data as it moves through each checkpoint.

Student	Running Total (Before Adding)	Added Score (This Entry)	Average (After Adding)
Alex	0 (no scores yet)	88 (Test 1)	88.00 (88 ÷ 1)
Alex	88 (from above)	92 (Test 2)	90.00 (180 ÷ 2)
Alex	180 (from above)	76 (Test 3)	109.33 (328 ÷ 3)

Investigate Here! This is the first wrong average. Check this row.

How to read this trace:
Each row shows how one score affects the running total and the new average. Find the first suspicious value, then check the calculation at that step.

Tip: The first wrong value is usually closest to the real cause. Check this row.

Demo 3: Grade Summary Debugging

26

Demo #4

"Order Total Debugging"

27



What to Watch For

Watch the calculation stages.

Useful checkpoints:

- ▶ line total
- ▶ subtotal
- ▶ discount
- ▶ tax
- ▶ final total

The bug is not just the wrong final number. The signal is the first wrong step.

28



Calculation Stages

Follow each checkpoint. The wrong value is first used in **Stage 3**.

Scenario	1 Line Total	2 Subtotal	3 Discount	4 Tax	5 Final Total
Items total	\$200.00	\$200.00	\$220.00	\$17.60	\$237.60
Discount	10%	Sum of all items.	Should subtract 10% of \$200 (\$20.00), not add it.	8% of \$220.00 (Based on the wrong amount).	Amount due (after tax).
Tax rate	8%	Looks good	Wrong value! Not used here.	Carries error forward	Final result is also wrong

How to use this:
Check each stage in order. The first stage with a wrong value is usually where the bug is. Fix that stage, and the later stages should fix themselves.

Investigate Here
The discount should be a subtraction, not an addition.

Demo 4: Order Total Debugging

29



Evidence Before Repair

Good debugging evidence narrows the problem.

Evidence may be:

- ▶ a traceback line
- ▶ expected versus actual output
- ▶ a labeled debug print
- ▶ the first wrong value
- ▶ a simple check after the fix

Do not change several things before you know what the evidence says.

30

Common Failure: Changing Code First

Changing code before observing can destroy the trail.

Instead:

- ▶ reproduce the problem
- ▶ record what happened
- ▶ inspect one clue
- ▶ change one thing
- ▶ run again

31

A Simple, Focused Debugging Loop

One step at a time. Small changes. Clear answers.



Common Failure: Changing Code First

32

Assignment 6

For Assignment 6, repair broken code and explain the debugging process.

Your work should show:

- ▶ the issue you found
- ▶ the evidence that helped
- ▶ the fix you made
- ▶ the check that proves the fix works

33

DEBUGGING NOTES

Capture the facts. Make one change. Verify the result.

1 EXPECTED What did you expect to happen?	
2 ACTUAL What actually happened?	
3 EVIDENCE What output, message, or value drove what happened?	
4 FIX What one change will you make?	
5 CHECK AFTER FIX What do you expect to see now? How will you verify it works?	

Debugging Notes Template

34

Evidence For A Debugging Submission

Useful debugging evidence includes:

- ▶ corrected '.py' file
- ▶ expected-vs-actual comparison
- ▶ labeled debug output
- ▶ traceback clue
- ▶ simple check or test case
- ▶ explanation of why the fix works

The evidence should support the repair, not just decorate the submission.

35

DEBUGGING EVIDENCE

One change. Clear evidence. Problem understood.

1. Corrected File: order_total.py <pre> 1 def calculate_total(subtotal, discount_rate, tax_rate): 2 discount = subtotal * discount_rate 3 after_discount = subtotal - discount 4 tax = after_discount * tax_rate 5 total = after_discount + tax 6 return total 7 8 # Example run 9 total = calculate_total(100, 0.20, 0.10) 10 print(f'Total: {total}') </pre> Change made: Subtract discount instead of adding it.	2. Expected vs Actual <table border="1"> <tr> <th>EXPECTED</th> <th>ACTUAL (Before Fix)</th> </tr> <tr> <td>80.00</td> <td>120.00</td> </tr> </table> Correct total after 20% discount and 10% tax. This happened when the discount was added instead of subtracted.	EXPECTED	ACTUAL (Before Fix)	80.00	120.00
EXPECTED	ACTUAL (Before Fix)				
80.00	120.00				
3. Labeled Debug Output (Before Fix) <pre> subtotal: 100.00 discount_rate: 0.20 discount: 20.00 after_discount: 80.00 tax: 12.00 total: 92.00 </pre> These prints show the first suspicious value.	4. Traceback Clue <pre> Traceback (most recent call last): File "order_total.py", line 9, in calculate_total after_discount = subtotal + discount TypeError: unsupported operand type(s) for +: 'float' and 'float' </pre> Clue: Line 9 is where the incorrect operation happens.				
5. Explanation & Fix The discount was added instead of subtracted on line 3. This made the after_discount too large, which caused the tax and total to be too large as well. Fix: Change 'subtotal + discount' to 'subtotal - discount'.	6. After the Fix After the fix: Total: 80.00 (tax expected)				

Evidence

36

A photograph of laboratory glassware, including a beaker and a graduated cylinder, both containing a blue liquid. The background is a blurred laboratory setting.

 Southwest
Tech

Closing Success Check

If you found the first useful
signal, you are debugging.

Before submitting Assignment #6,
ask:

- ▶ What was wrong?
- ▶ What evidence showed it?
- ▶ What did I change?
- ▶ How did I check the fix?
- ▶ Can I explain the repair
without guessing?

37

A large blue thumbs-up icon.

 Southwest
Tech

Thank you! Have Fun!

38